BİTİRME PROJESİ · ARA SUNUM

minGPT'den FPGA'ye

Bir dil modeli ailesini yalnızca tamsayı aritmetiğiyle, küçük bir FPGA üzerinde, bit bit doğru çalıştırmak

Murat Atceken

PYNQ-Z2 · Zynq XC7Z020 · Ekim 2026

GİRİŞ

Proje bir cümlede

Karpathy’nin minGPT’sini ve aynı soydan dokuz dil modelini, hiç kayan nokta kullanmayan tek bir FPGA motoruna taşıdık; RTL simülasyonunda hepsi yazılım referansıyla *bit bit* aynı sonucu veriyor.

729/729

minGPT sıralama görevinde doğru çözülen dizi (tüm girdi uzayı)

10 model

tek motor, tek bitstream: bigram’dan Llama’ya ve stories260K’ya

0 hata

rastgele testte 1 256 707 logitin hepsi referansla aynı

145 µs

minGPT’de token başına süre (14 544 çevrim, 100 MHz hedefi)

GİRİŞ

Yol haritası

1 **Teorik zemin**
Dil modeli nedir, nasıl ölçülür, nasıl eğitilir

2 **minGPT adım adım**
Her adım: formül, sezgi ve onu yapan kod

3 **Tamsayı aritmetiği**
Nicemleme, tamsayı LayerNorm ve softmax

4 **İlk donanım**
gpt_top: minGPT'ye özel hızlandırıcı

5 **Model ailesi ve motor**
Dokuz mimari, tek mikrokodlu motor

6 **Doğrulama ve sonuçlar**
Bit-exact zincir, çevrim modeli, ölçümler

7 **Gerçekten anladık mı?**
Beş temel soru, beş gerekçeli cevap

8 **Neredeyiz**
Kalan işler, sınırlar, kart ve makale

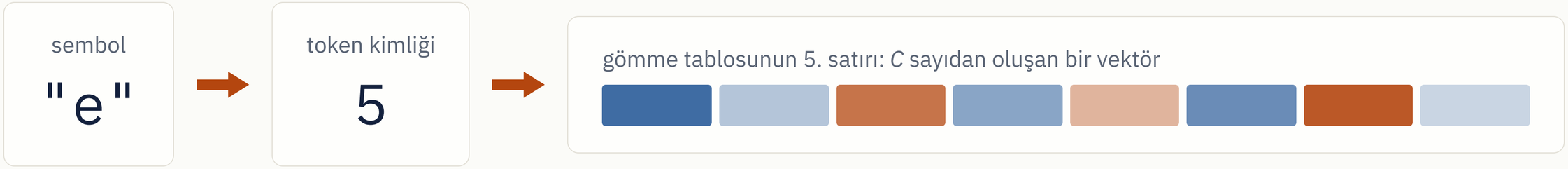
Dil modeli: bir sonraki sembolün olasılığı

$$p(x_1, \ldots, x_n) = \prod_t p(x_t \mid x_1, \ldots, x_{t-1})$$

- Olasılığın zincir kuralı: yaklaşıklık değil, tam bir eşitlik.
- Model tek bir şey öğrenir: geçmişe bakıp bir sonraki sembol için bir dağılım vermek.
- Üretim: dağılımdan seç, sona ekle, tekrarla (otoregresif).



Token ve gömme: harften sayıya, sayıdan vektöre



$$\mathbf{e}(\tau) = W_{\text{te}}[\tau] = \text{onehot}(\tau)^{\intercal}\, W_{\text{te}}\ ,\ W_{\text{te}} \in \mathbb{R}^{V \times C}$$

- İsim verisi: $V = 27$ (26 harf ve bitiş işareti “.” = 0, a = 1, …).
- Gömme bir matris çarpımıdır, ama tek bir satırı okumaya eşittir. Donanımda bu, bir ROM adresi demek.

minGPT · model.py

```
wte = nn.Embedding(vocab_size, n_embd)
wpe = nn.Embedding(block_size, n_embd)
# forward içinde:
tok_emb = self.transformer.wte(idx)
```

Ölçü ve eğitim: çapraz entropi ve AdamW

$$\mathcal{L} = -\,(1/N)\,\Sigma_t \ln p(x_t \mid x_1, \ldots, x_{t-1})$$

Doğru tokena verilen olasılığın eksi logaritması; birimi **nat/token**. Küçük olan iyi.



$e^{1.96} \approx 7$: model, 27 harf yerine ortalama ~7 aday arasında kararsız.

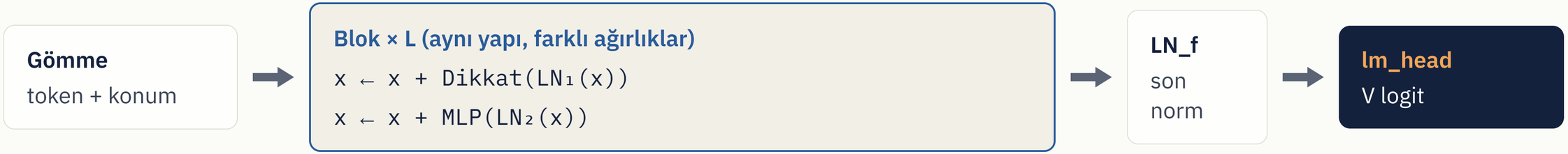
minGPT · trainer.py (eğitim döngüsü)

```
logits, self.loss = model(x, y)
model.zero_grad(set_to_none=True)
self.loss.backward()
clip_grad_norm_(model.parameters(), 1.0)
self.optimizer.step() # AdamW
```

$$\theta \leftarrow \theta - \eta\,(\,\hat{m} / (\sqrt{\hat{v}} + \varepsilon) + \lambda\theta\,)$$

m, v: gradyanın ve karesinin kayan ortalaması; λ: ağırlık çürümesi

minGPT’nin bütünü tek bakışta



minGPT · GPT.forward (model.py)

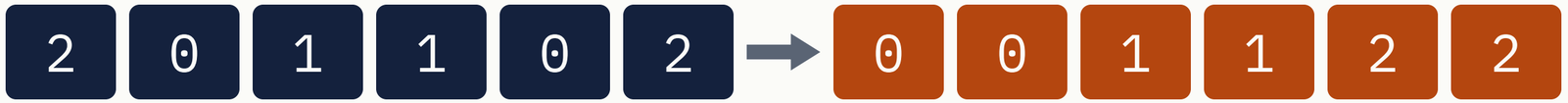
```
tok_emb = self.transformer.wte(idx)
pos_emb = self.transformer.wpe(pos)
x = self.transformer.drop(tok_emb + pos_emb)
for block in self.transformer.h:
    x = block(x)
x = self.transformer.ln_f(x)
logits = self.lm_head(x)
```

gpt-nano (sıralama görevi)

$L = 3$ blok · $H = 3$ baş
$C = 48$ genişlik · $d_h = 16$
$V = 3$ sembol · $T = 11$ bağlam

≈ 85,7 bin parametre

İlk görev: altı rakamı sıralamak



Lacivert: girdi (6 rakam, {0, 1, 2}) · Turuncu: modelin üretmesi gereken sıralı çıktı

minGPT · demo.ipynb, SortDataset

```
sol = torch.sort(inp)[0]
cat = torch.cat((inp, sol), dim=0)
x = cat[:-1].clone() # 11 token girer
y = cat[1:].clone() # bir kaydırılmış hedef
y[:self.length-1] = -1 # girdi kısmında kayıp yok
```

$3^6 = 729$

olası girdinin hepsi: test kümesi bütün uzay

$729 \times 11 = 8\,019$

donanımda karşılaştırılan adım (token)

Adım 1 · Gömme: “ne” ve “nerede” toplanır

$$x_t^{(0)} = W_{te}[\tau_t] + W_{pe}[t] \quad \tau_t\text{: } t \text{ konumundaki token}$$

Konum neden gerekli?

Dikkat, değerlerin ağırlıklı toplamıdır; toplama sıra bilmez. Konum vektörü olmasa “2 0 1” ile “1 0 2” modele aynı görünürdü.

Dropout yalnızca eğitimde etkin; çıkarımda birim fonksiyon. Donanımda yoktur.

minGPT · GPT.forward

```
pos = torch.arange(0, t, dtype=torch.long)
tok_emb = self.transformer.wte(idx) # (b, t, C)
pos_emb = self.transformer.wpe(pos) # (1, t, C)
x = self.transformer.drop(tok_emb + pos_emb)
```

Adım 2 · LayerNorm: her vektörü aynı ölçeğe getirmek

$$\mu = (1/C)\,\Sigma_i\, x_i\,,\ \sigma^2 = (1/C)\,\Sigma_i\,(x_i - \mu)^2$$
$$\mathrm{LN}(x) = \gamma \odot (x - \mu) \,/\, \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

Küçük örnek

$x = [2, 4, 6] \rightarrow \mu = 4,\ \sigma \approx 1{,}633$
$\rightarrow (x - \mu)/\sigma \approx [-1{,}22,\ 0,\ 1{,}22]$

Ortalama 0, varyans 1. γ ve β öğrenilir. Donanım için zor kısım: karekök ve bölme.

minGPT · Block

```
self.ln_1 = nn.LayerNorm(config.n_embd)
self.ln_2 = nn.LayerNorm(config.n_embd)
# pre-LN: norm, alt katmandan önce
x = x + self.attn(self.ln_1(x))
```

Adım 3 · Sorgu, anahtar, değer: üç doğrusal izdüşüm

$$[q_t\ ;\ k_t\ ;\ v_t] = W_{attn} \cdot \mathrm{LN}_1(x_t) + b\ ,\ W_{attn} \in \mathbb{R}^{3c \times c}$$

q · sorgu

“Ben ne arıyorum?”

k · anahtar

“Bende ne var?”

v · değer

“Seçilirsem ne aktarırım?”

minGPT · CausalSelfAttention

```
self.c_attn = nn.Linear(n_embd, 3 * n_embd)
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
```

Donanım açısından

Bir Linear = matris-vektör çarpımı (GEMV): $y_i = \Sigma_j W_{ij} x_j + b_i$. gpt-nano’da bu satır 144 × 48 = 6 912 çarpma-toplama.

Adım 4 · Ölçekli nokta çarpım dikkati

$$s_{t\,j} = q_t \cdot k_j \,/\, \sqrt{d_h}\,,\; j \le t$$

1 · benzerlik skoru

$$p_{t\,j} = \exp(s_{t\,j}) \,/\, \Sigma_{i \le t} \exp(s_{t_i})$$

2 · softmax: olasılığa çevir

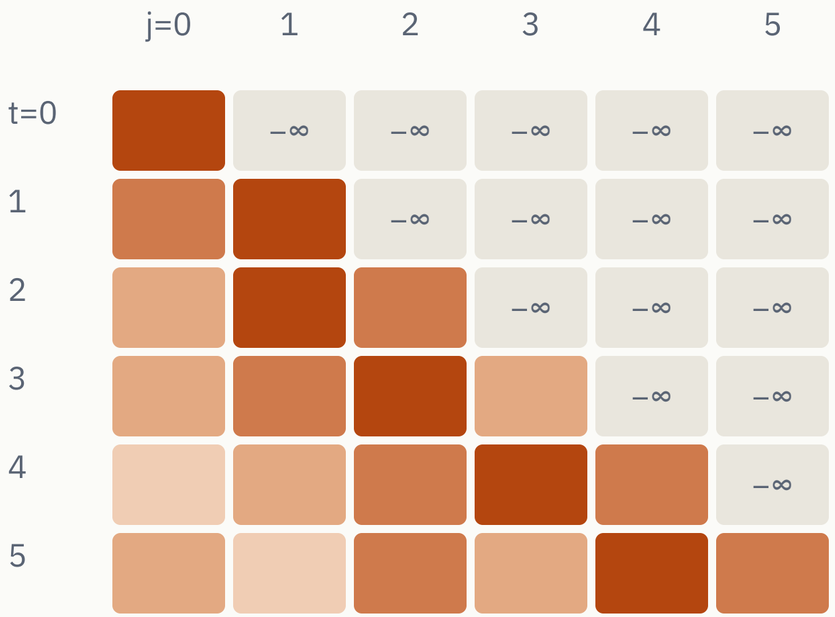
$$y_t = \Sigma_{j \le t}\, p_{t\,j}\; v_j$$

3 · değerlerin ağırlıklı ortalaması

minGPT · CausalSelfAttention.forward

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1))) # 1
att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf')) # maske
att = F.softmax(att, dim=-1) # 2
y = att @ v # 3
```

Nedensel maske: gelecek görülmez



Satır: sorgu konumu *t* · sütun: anahtar konumu *j* · renk tonları temsilî

−∞ eklemek, softmax'ta exp(−∞) = 0 demektir: gelecekteki tokenın ağırlığı tam olarak sıfır.

Her satır bir olasılık dağılımıdır: renkli hücrelerin toplamı 1.

Sonuç: *t* konumunun çıktısı yalnızca 0…*t* tokenlarına bağlı. KV önbelleği bu gerçeğe dayanır.

```
torch.tril(torch.ones(T, T)) # alt üçgen
```

Adım 5 · Çok başlı dikkat: aynı anda birkaç soru



$$y = W_o\,[\,\text{baş}_1\,;\,\text{baş}_2\,;\,\text{baş}_3\,] + b$$

Her baş kendi 16 boyutlu alt uzayında ayrı bir dikkat dağılımı kurar. Toplam çarpma sayısı tek başlı dikkatle aynı.

minGPT · CausalSelfAttention.forward

```
k = k.view(B, T, nh, C // nh).transpose(1, 2)
# ... her baş için dikkat ...
y = y.transpose(1, 2).contiguous().view(B, T, C)
y = self.resid_dropout(self.c_proj(y))
```

Adım 6 · Artık bağlantı: bilginin ana yolu

$$x \leftarrow x + f(\mathrm{LN}(x))$$

$$\partial x_{out} / \partial x_{in} = I + \partial f / \partial x$$

- Her alt katman vektörü baştan yazmaz; üstüne bir **düzeltme ekler**.
- Birim matris I sayesinde gradyan derin ağda sönmeden geri akar.
- Bütün katmanlar aynı “artık akıştan” okur ve ona yazar.

minGPT · Block.forward (tamamı)

```
def forward(self, x):
    x = x + self.attn(self.ln_1(x))
    x = x + self.mlpf(self.ln_2(x))
    return x
```

Donanımdaki karşılığı

Artık akış INT16 ve ölçeği 2’nin kuvveti: toplama, ölçek dönüşümü olmadan doğrudan tamsayı toplamadır.

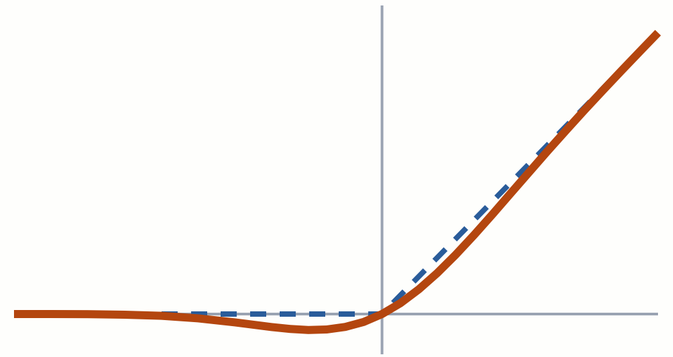
Adım 7 · MLP ve GELU: her token kendi içinde

$$\mathrm{MLP}(a) = W_2\,\mathrm{GELU}(W_1\,a + b_1) + b_2$$

$$\mathrm{GELU}(x) = \tfrac{1}{2}\,x\,(1 + \tanh(\sqrt{2/\pi}\,(x + 0{,}044715\,x^3)))$$

minGPT · Block.__init__

```
c_fc = nn.Linear(n_embd, 4 * n_embd), # 48 → 192
c_proj = nn.Linear(4 * n_embd, n_embd), # 192 → 48
act = NewGELU(),
self.mlpf = lambda x: m.dropout(m.c_proj(m.act(m.c_fc(x))))
```



GELU (düz) ve **ReLU** (kesikli), $x \in [-4, 3]$

Negatif tarafta yumuşak; sıfır çevresinde türevlenebilir.

Adım 8 · Çıkış ve üretim: logitten tokena

$$z_t = \text{lm_head}(\,\text{ln_f}(x_t)\,) \in \mathbb{R}^V$$

$$\tau_{t+1} = \text{argmax}_i\ \text{softmax}(z_t)_i = \text{argmax}_i\ z_{t_i}$$

- Softmax monoton artan: en büyük olasılık en büyük logittedir. Açgözlü üretimde softmax gereksiz.
- minGPT her adımda **bütün diziyi** baştan hesaplar. Donanım bunu yapmaz (sonraki slayt).

minGPT · GPT.generate (döngü gövdesi)

```
logits, _ = self(idx_cond) # tüm önek!
logits = logits[:, -1, :] / temperature
probs = F.softmax(logits, dim=-1)
if do_sample:
    idx_next = torch.multinomial(probs, 1)
else:
    _, idx_next = torch.topk(probs, k=1, dim=-1)
idx = torch.cat((idx, idx_next), dim=1)
```

KV önbelleği: her adımda yalnızca yeni token

> **Önerme.** Nedensel maske altında j konumundaki her ara değer (x_j, k_j, v_j, her katmanda) yalnızca $\tau_0 \dots \tau_j$'ye bağlıdır.
>
> İspat: katman üzerinden tümevarım. Gömme, LN, Linear ve GELU konum başınadır; dikkat yalnızca $i \le j$ konumlarını okur. ∎

gpt-nano, token başına	boyut	MAC
q, k, v (c_attn)	144 × 48	6 912
c_proj · c_fc · mlp.c_proj	—	20 736
3 blok + lm_head	—	**83 088**
Dikkat, katman başına	$2C(t{+}1)$	96 … 1 056

Bizim altın model · sw/intmodel.py, step(tok, t)

```
qkv = linear(ql["attn"], z, 8)
qv = qkv[:C]
self.kc[l][t] = qkv[C:2 * C] # K sakla
self.vc[l][t] = qkv[2 * C:] # V sakla
```

Sonuç: matris-matris çarpımı (GEMM) matris-vektöre (GEMV) iner; hesabın ~%97'si GEMV. Donanımın kalbi bu yüzden bir GEMV motoru.

3 · TAMSAYI ARİTMETİĞİ

FPGA nedir, neden tamsayı?

53 200
LUT
6 girişli tablo: herhangi bir mantık fonksiyonu

106 400
FF
tek bitlik yazmaç: durum ve boru hattı

140
BRAM (36 Kb)
çip içi bellek, toplam ~4,9 Mb

220
DSP48
25 × 18 bit tamsayı çarpıcı ve toplayıcı

PYNQ-Z2 kartı: Zynq XC7Z020

Tek çipte çift çekirdekli ARM Cortex-A9 (yazılım) ve yukarıdaki programlanabilir mantık (donanımımız).

Neden tamsayı?

- INT8 çarpma-toplama: bir DSP, her çevrimde bir tane.
- INT8 ağırlık float32'nin dörtte biri: model çip içine sığar.

Nicemleme: gerçel sayı = ölçek × tamsayı

$$q = \mathrm{clip}(\ \mathrm{round}(r\ /\ S),\ -128,\ 127\ )$$

$\hat{r} = S\,q$, aralık içinde $|r - \hat{r}| \le S/2$

Örnek: $S = 0{,}02$, $r = 0{,}537 \rightarrow r/S = 26{,}85 \rightarrow q = 27 \rightarrow \hat{r} = 0{,}54$; hata $0{,}003 \le 0{,}01$.

Tensör	Biçim	Ölçek
Ağırlıklar	INT8	çıkış kanalı başına, $\max\|W_o\|/127$
Aktivasyonlar	INT8	tensör başına, kalibrasyondan
Artık akış	INT16	2^{-12} (Q3.12), 2'nin kuvveti

sw/intmodel.py · doyma (clip)

```
def sat(v, bits):
    lo = -(1 << (bits - 1))
    hi = (1 << (bits - 1)) - 1
    return max(lo, min(hi, v))
```

Her ek bit, sinyal/nicemleme gürültüsü oranını ≈ 6,02 dB artırır.

Tamsayı Linear: çarp, topla, yeniden ölçekle

$$y_o \approx S_w[o]\, S_x \cdot \mathrm{acc}_o \,,\ \mathrm{acc}_o = \Sigma_i\, q^w_{oi}\, q^x_i + q^b_o$$

tam sayı birikim (INT32)

$$M_o = S_w[o]\, S_x \,/\, S_y \approx m0 \cdot 2^{-sh} \,,\ m0 \in [2^{15}, 2^{16})$$

bağıl hata ≤ 2^{-16}

$$q^y_o = \mathrm{sat}(\ (\mathrm{acc}_o \cdot m0 + 2^{sh-1}) \gg sh\ )$$

kaydırma = bölme, +yarım = yuvarlama

sw/intmodel.py · altın model (RTL ile birebir)

```
def requant(acc, m0, sh, bits):
    return sat((acc * m0 + (1 << (sh - 1))) >> sh, bits)
# linear():
accs = W @ np.asarray(x, dtype=np.int64)
out.append(requant(acc + b[o], m0[o], sh[o], bits))
```

Taşma olmaz

$|\mathrm{acc}| \le 192 \cdot 127 \cdot 128 + |q^b| \approx 3{,}1 \cdot 10^6 < 2^{22}$. INT32 birikimci hiçbir girdide taşmaz.

Katlama: donanıma girmeden işi azaltmak

LayerNorm’un γ ve β’sı

$$W(\gamma \odot \hat{z} + \beta) + b = (W \operatorname{diag} \gamma)\,\hat{z} + (W\beta + b)$$

LN’den sonra hep bir Linear gelir. Donanımdaki LN yalnızca $\hat{z}$’yi hesaplar; γ, β için bellek ve çarpıcı gerekmez.

Dikkatin $1/\sqrt{d_h}$ ölçeği

$$(q \cdot k) \,/\, \sqrt{d_h} = (q \,/\, \sqrt{d_h}) \cdot k$$

$1/\sqrt{16} = 1/4$ çarpanı *W*_q ve *b*_q satırlarına gömülür. ln_f de lm_head’e katlanır.

sw/zoo/ir.py · derleyici (sadeleştirilmiş)

```
def fold_affine(W, b, gamma, beta):
    W2 = W * gamma[None, :]
    b2 = b + W @ beta
    return W2, b2
Wq, bq = Wq / math.sqrt(dh), bq / math.sqrt(dh)
```

Kontrol

Katlanmış float64 model, minGPT ile 729 dizinin hepsinde 10^{-4} toleransla aynı logitleri verir. Bunu kod her seferinde otomatik denetler.

Tamsayı LayerNorm: kayan noktasız karekök ve bölme

$$d_i = N\,x_i - \Sigma_j\, x_j = N(x_i - \mu) \cdot Q = \Sigma_i\, d_i^{\,2} + E \cdot rt = \lfloor \sqrt{Q} \rfloor \cdot inv = \lfloor K \,/\, rt \rfloor$$

$$z_i = \mathrm{sat}_8(\,(d_i \cdot inv + \text{yarım}) \gg (P - ZF)\,)\;,\; K = \mathrm{round}(\sqrt{N} \cdot 2^{P})\;,\; E = \varepsilon\text{'un tamsayı karşılığı}$$

sw/intmodel.py · IntGPT.layernorm (gövde)

```
s = sum(x)
d = [N * xi - s for xi in x] # N(x − μ), tam
Q = sum(di * di for di in d) + q["LN_E"]
rt = math.isqrt(Q) # ⌊√Q⌋
inv = q["LN_K"] // rt # tek bölme
sh = q["LN_P"] - q["ZF"]
return [sat((di * inv + (1 << (sh - 1))) >> sh, 8) for di in d]
```

Doyma yok (ispatlı)

Samuelson: $|\hat{z}_i| \le \sqrt{(N-1)} = \sqrt{47} \approx 6{,}86 < 7{,}94$. INT8 Q3.4 çıkış hiçbir girdide taşmaz.

Maliyet

LN başına bir karekök (32 çevrim) ve bir bölme (48 çevrim); kalanı akış hâlinde.

Tamsayı softmax: üç matematiksel hamle

1 · Kaydır

$\mathrm{softmax}(s) = \mathrm{softmax}(s - \max s)$
→ üsler ≤ 0, $e_j \in (0, 1]$

2 · 2 tabanı ve tablo

$e^{-x} = 2^{-n} \cdot 2^{\wedge}(-f/2^{f})$
kaydırma × 256 girişlik tablo

3 · Bölmeyi ertele

$y = (\Sigma_j\ e_j\ v_j) / (\Sigma_j\ e_j)$
baş başına tek bölme

sw/intmodel.py · attention_head (sadeleştirilmiş)

```
u = (m - sj) * ql["CEXP"] # (m − s)·log₂e ≥ 0
n, k = u >> F, (u >> (F - 8)) & 0xFF
e.append(0 if n > 15 else EXP_LUT[k] >> n)
R = (1 << rs) // sum(e) # tek bölme
y.append(sat((A * R + (1 << (rs - 1))) >> rs, 8))
```

Hata sınırı

Tablonun 8 bit kesirle adreslenmesi: e_j'de < %0,27 bağıl hata.

Doyma yok

Çıkış, v'lerin dışbükey birleşimi: $\min v \le y \le \max v$.

GELU tablosu ve argmax: yaklaşıklık gerektirmeyenler

GELU: tam tablo

$$\mathrm{LUT}[q] = \mathrm{round}\left(\mathrm{GELU}(q\, S_\mathrm{h}) \,/\, S_\mathrm{g} \right), \quad q \in [-128, 127]$$

Giriş 8 bit olduğundan yalnızca 256 değer alır: tablo fonksiyonu **tam** temsil eder. Katman başına 256 bayt.

```
g = [ql["GELU_LUT"][v + 128] for v in hq]
```

Argmax: softmax’sız

$$\mathrm{argmax}\ \mathrm{softmax}(z) = \mathrm{argmax}\ z$$

Logitler ortak bir ölçekte INT16: tamsayı karşılaştırma yeterli. Eşitlikte en küçük indeks; yazılım da aynı kuralı uygular.

```
max(range(V), key=lambda i: (logits[i], -i))
```

Tamsayı model, gpt-nano **729/729** doğru float ile token uyumu **%100** logit SQNR **41,5 dB**

gpt_top: forward() bir durum makinesine dönüşür



Durum	Donanım birimi	minGPT'deki karşılığı
EMB	iki ROM okuması, doymalı INT16 toplama	wte(idx) + wpe(pos)
LN1, LN2, LNF	layernorm (afin kısım katlanmış)	ln_1, ln_2, ln_f
QKV	gemv 48 → 144, K ve V önbelleğe	c_attn(x).split(...)
ATT	attention, 3 baş ardışık	softmax(q·k^T) @ v
PROJ, MPROJ	gemv, artık akışa doymalı ekleme	x = x + c_proj(...)
FC	gemv 48 → 192, çıkış GELU tablosundan	act(c_fc(x))
HEAD	gemv 48 → 3, akış hâlinde argmax	lm_head(x), topk(k=1)

GEMV motoru: 8 şerit, çıkış-durgun, boşluksuz

her çevrim

x_i

yayın

64 bit ağırlık kelimesi: 8 çıkışın aynı *i* sütunu

$W_{0\,i}$	$W_{1\,i}$	$W_{2\,i}$	$W_{3\,i}$	$W_{4\,i}$	$W_{5\,i}$	$W_{6\,i}$	$W_{7\,i}$
acc_0	acc_1	acc_2	acc_3	acc_4	acc_5	acc_6	acc_7

8 MAC şeridi: acc_p += $W_{pi} \cdot x_i$ (INT8 × INT8 → INT32)

gölge banka

→ requant, çevrim başına bir çıkış; şeritler bu arada sonraki 8 çıkışa başlar

rtl/gemv.sv (SystemVerilog)

```
for (int p = 0; p < P; p++) begin
  prod[p] = $signed(w_word[8*p +: 8]) * x1;
  acc_nx[p] = (first1 ? 32'sd0 : acc[p]) + prod[p];
end
if (handoff) acc_sh[p] <= acc_nx[p]; // gölge banka
```

$(n_{out} / P) \cdot n_{in}$ + 14 çevrim

bir Linear'ın süresi (ölçüldü)

%98,3 verim

10 416 ideal MAC çevrimine karşı 10 598

4 · İLK DONANIM

İlk sonuç: doğru, hızlı ve küçük

8 019/8 019 — adım RTL'de bit-exact (Verilator); 729/729 dizi doğru

14 544 — çevrim/token → 100 MHz hedefinde 145 µs

%73 — sürenin GEMV'de geçen kısmı; LN %11, dikkat %16

Kaynak kullanımı (Yosys, XC7Z020)



Vivado zamanlama ve kart ölçümü henüz yok.

5 · MODEL AİLESİ VE MOTOR

Her mimari fikir donanımda neye mal olur?

Eşit parametre bütçesi, aynı veri, aynı tamsayı motoru. Böylece kalite ve maliyet aynı ölçekte yan yana konabilir.

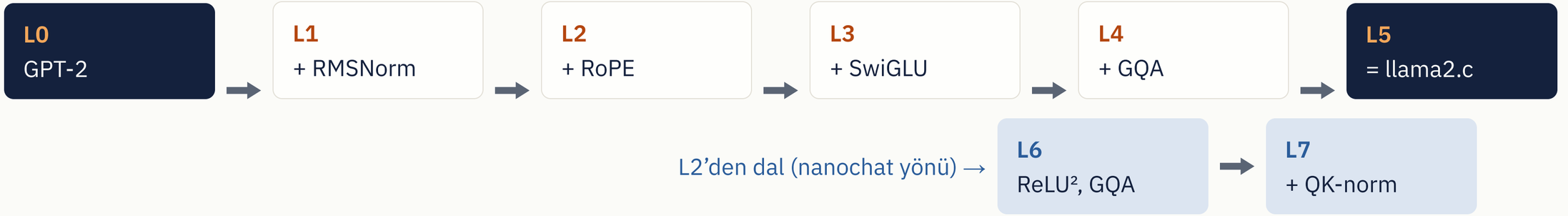
5 · MODEL AİLESİ VE MOTOR

Dokuz mimari, 50 bin parametre, isim verisi

Model	Kaynak	Ayırt edici fikir	Test kaybı	Δ INT8	Çevrim/token
Bigram	makemore	önceki harfe göre tablo	2,451	0,000	36
MLP	makemore	sabit pencere, gizli katman	1,986	+0,001	6 632
RNN	makemore	tekrarlayan gizli durum	1,964	+0,003	6 374
GRU	makemore	kapılı tekrarlayan durum	**1,956 (en iyi)**	+0,000	6 515
BoW	makemore	önek ortalaması	2,045	−0,000	6 277
GPT-2	minGPT	LayerNorm, öğrenilen konum, GELU	2,000	+0,002	8 620
microgpt	microgpt (2026)	parametresiz RMSNorm, ReLU	1,971	+0,003	8 657
Llama 2	llama2.c	RMSNorm, RoPE, SwiGLU, GQA	2,003	+0,001	9 396
nanochat	nanochat	RoPE, QK-norm, ReLU², GQA	1,969	+0,006	10 711

Ön sonuç: tek tohum, öğrenme oranı taraması sürüyor. Kayıp nat/token (1000 isimlik test kümesi). Bigram 729 parametre.

Özellik merdiveni: her basamakta tek değişiklik



Neden böyle?

Ailedeki modeller birden çok yönden farklı. Merdivende her basamak tek bir kararı değiştirir; kayıp ve çevrim farkı o fikre atfedilebilir. Bütçe, MLP genişliği aranarak eşit tutulur.

Durum

Eğitim kampanyası sürüyor (isimler: 51 koşudan 20'si tamam). Merdivenin sonuçları henüz yok.

Mikrokodlu motor: her model bir program

PyTorch modeli → ir.py: derle → float program = PyTorch → quant.py: tamsayı program → .mem görüntüleri

Sıralayıcı

256 bitlik komutları sırayla okur, ilgili birimi başlatır, bitmesini bekler.

Bellekler: R (artık, INT16), A…H vektörler, K/V önbelleği, HS (tekrarlayan durum)

GEMV · *P* şerit
o8, artığa ekle, tablo aktivasyonu, logit

NORM
LayerNorm ve RMSNorm, segment başına

ATTN · 8 şerit
tamsayı softmax, GQA, KV önbelleği

ROPE
döner konum, iki eşleme stili

EMB
token, konum, pencere tabloları

Eleman bazlı
MUL, GRU güncelleme, BoW, kopya

Derleme parametreleri: SPECIALIZE (yalnızca kullanılan birimler), LOADABLE (bellekler çalışırken yüklenir), PRELOAD.

15 işlemlik komut kümesi ve bir GPT-2 programı

Gömme ve çıkış

EMB16 · EMB8 · EMBWIN · TABLOGIT

Ana hesap

NORM · GEMV · ATTN · ROPE · KVSTORE

Eleman bazlı ve durum

MUL · GRUUPD · COPY · REQ · BOW · HRESET

> **GEMV modları**
>
> o8: INT8 çıkış · radd: artığa ekle · lut: tablo aktivasyonu (GELU, SiLU…) · logit: INT16

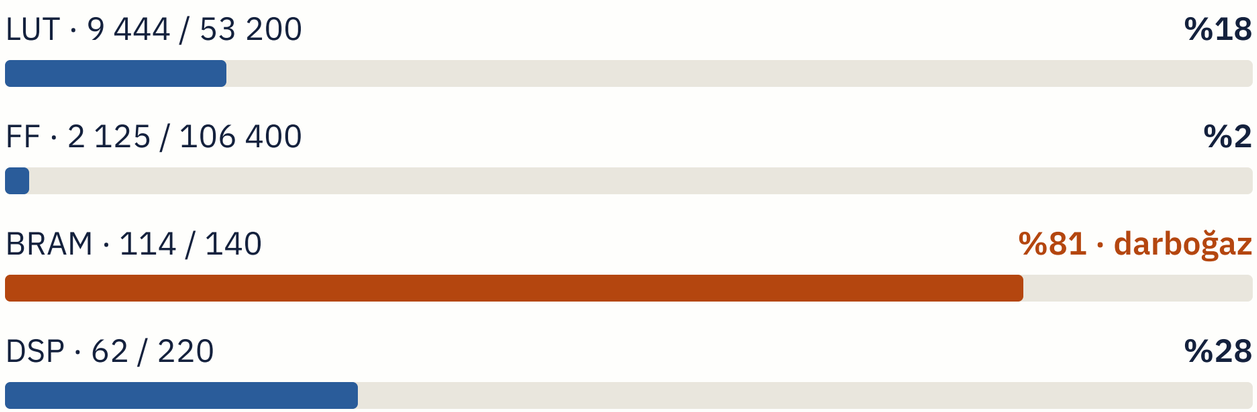
Program uzunluğu: RNN 6, GRU 9, GPT-2 44, Llama 60, nanochat 61 komut.

GPT-2 (isimler), derleyici çıktısı

```
EMB16   wte + wpe → R
NORM    ln: R → A
GEMV    l0.q, l0.k, l0.v: A → Q, K, V (o8)
KVSTORE l0: K, V → önbellek
ATTN    l0: Q → B (4 baş, dₕ = 8)
GEMV    l0.proj: B → R (radd)
NORM    ln: R → A
GEMV    l0.fc: A → F (lut, gelu)
GEMV    l0.mproj: F → R (radd)
...     katman 1–3 aynı
NORM    ln: R → A
GEMV    head: A → L (logit)
```

Tek bitstream: on model, tek donanım, AXI ile yükleme

Evrensel motor (*V* = 512, *T* = 256, 5 katman) · Yosys



BoW’un 21 bitlik tablosu iki 16 bitlik kelimeye bölündü: BRAM 132 → 114.

board/pynq_z2/zoo.py · bir adım

```
def step(self, tok, pos):
  w(REG["TOK"], tok)
  w(REG["POS"], pos)
  w(REG["CTRL"], 1)
  while not r(REG["STATUS"]) & 2:
    pass
  return r(REG["ARGMAX"]),
         r(REG["CYCLES"])
```

Model yükleme: 256 bitlik pencere (LD_SEL, LD_ADDR, LD_DATA, LD_GO).

AXI tezgâhında 10 modelin hepsi ve 200 tokenlık kapalı döngü hikâye üretimi bit-exact. Kartta henüz çalıştırılmadı.

Doğrulama zinciri: her halka bir öncekine eşit



1 256 707

logit, 60 rastgele vakada (rastgele model, rastgele girdi): **0 hata**

36,4 milyon

bölücü ve karekök vakası; küçük bölücüde tüm uzay, tanım özelliğiyle denetim

10 model

C simülatörü ve RTL bit-exact; Icarus'ta X (belirsiz bit) yayılımı yok

Kapalı biçimli çevrim modeli: simülasyonsuz maliyet

$$C(t) = 3 + 2N_{\text{komut}} + \Sigma_{\text{norm}}\, S(2n + 73) + \Sigma\text{GEMV}\,[\, n_{\text{in}} + (B-1)\cdot\max(n_{\text{in}}, P+2) + P + 5\,]$$
$$+ \Sigma\text{ATTN}\; H(2(t+1) + 30) + C_\text{diğer}$$

NORM

n toplama, 2 hazırlık, 29 karekök, 42 bölme, n çıkış

GEMV

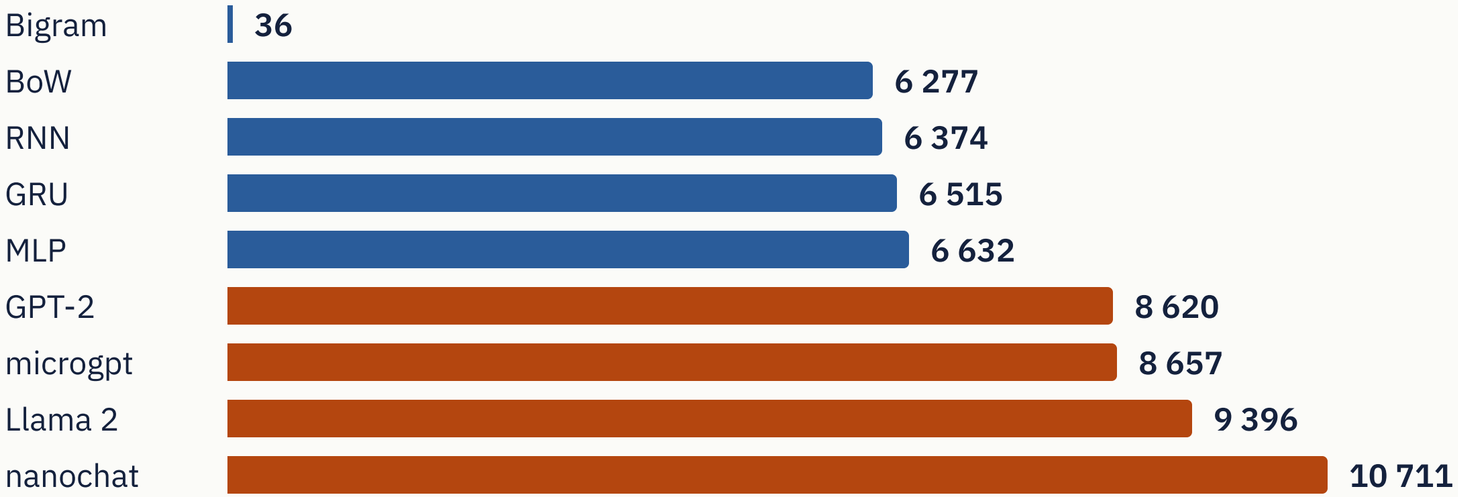
blok başına max(birikim n_{in}, seri requant P+2); $B = \lceil n_{\text{out}}/P \rceil$

ATTN

baş başına t+1 skor, t+1 üs, 19 karşılık, 8 çıkış, 3 geçiş

Bütün modellerde ve bütün konumlarda simülasyonla **çevrimi çevrimine aynı**. Bir mimarinin maliyeti, program derlenir derlenmez okunur.

Eşit bütçe, eşit GEMV: fark normalizasyon ve dikkatte



Çevrim/token, isimler, 50k · **mavi: tekrarlayan, MLP, BoW** · **turuncu: transformer**

5 900–6 700

GEMV çevrimi, her modelde. Eşit parametre ≈ eşit çarpma.

+1 233 … +3 506

NORM çevrimi, transformerlarda (nanochat'te QK-norm ve ek normlar)

+613

ATTN, ortalama; Llama'da RoPE +336, SwiGLU çarpımı +396

Gerçek bir model: stories260K, 512 tokenlık bağlam

Çevrimin dağılımı: kısa bağlam ve uzun bağlam

GPT-2, isimler ($T \le 16$) · 8 620 çevrim



stories260K ($T = 512$) · ortalama 59 092 çevrim



mavi GEMV · **turuncu dikkat** · **lacivert NORM** · gri diğer

Dikkatin payı %7’den %37’ye çıkıyor: çevrim, konumla doğrusal artıyor (38 652 → 79 532).

1,400 → 1,422

float → INT8 test kaybı (Δ +0,022), argmax uyumu %92

200 / 200

RTL’nin ürettiği hikâyenin tokenları referansla aynı

0,052 → 0,022

kalibrasyon iyileştirmesiyle Δ; hata aktivasyonlardan (yalnız ağırlık: +0,001)

Tasarım uzayı: şerit sayısı $P = 8, 16, 32$

Çevrim/token	$P = 8$	$P = 16$	$P = 32$	8 → 32
MLP	6 632	3 480	2 056	**3,23×**
GRU	6 515	3 683	2 315	**2,81×**
GPT-2	8 620	5 684	4 540	1,90×
Llama 2	9 396	6 492	5 548	1,69×
nanochat	10 711	8 031	7 143	1,50×
stories260K	59 092	43 012	35 724	1,65×

18 noktanın hepsi RTL'de bit-exact. GPT-2 motoru (Yosys): LUT 7,3k → 8,2k → 10,0k; DSP 60 → 70 → 86.

Amdahl yasası, stories260K

GEMV payı $f = 0{,}562$ ve 3,37 kat hızlanıyor (33 209 → 9 841). Dikkat 8 şeritte, P'den bağımsız.

$$1 / ((1 - 0{,}562) + 0{,}562 / 3{,}37) \approx \mathbf{1{,}65\times}$$

Ölçülen değerle aynı. Uzun bağlamda bir sonraki hedef, GEMV değil dikkat birimi.

Dikkatte skorları neden $\sqrt{d_h}$'ye bölüyoruz?

Kısa cevap

Bölmezsek skorların yayılımı d_h ile büyür ve softmax tek bir tokena kilitlenir.

Gerekçe

q_i, k_i bağımsız, ortalama 0, varyans 1 ise
$\mathrm{Var}(q \cdot k) = \Sigma_i \mathrm{Var}(q_i\, k_i) = d_h \rightarrow \mathrm{std} = \sqrt{d_h}$
$\mathrm{Var}(q \cdot k \,/\, \sqrt{d_h}) = 1$, d_h'den bağımsız

Sayıyla: d_h = 16'da tipik skor farkı 4 kat büyür. 2 birimlik fark $e^2 \approx 7{,}4$ kat ağırlık oranı verirken 8 birimlik fark $e^8 \approx 2\,981$ kat verir: dağılım neredeyse tek noktaya çöker, gradyanlar söner.

Bizim tasarımda: $1/\sqrt{16} = 1/4$ çarpanı *W*_q'ya katlanır; donanımda bedavadır.

LayerNorm çıkışı neden hiçbir girdide taşmaz?

Kısa cevap

Normalize edilmiş bir eleman $\sqrt{(N-1)}$'i aşamaz. Bu, veriden bağımsız bir eşitsizlik.

İspat (Samuelson, 1968)

$$\Sigma_j (x_j - \mu) = 0 \Rightarrow x_i - \mu = -\Sigma_{j \neq i} (x_j - \mu)$$

Cauchy–Schwarz: $(x_i - \mu)^2 \leq (N-1) \Sigma_{j \neq i} (x_j - \mu)^2$

$= (N-1) (N\sigma^2 - (x_i - \mu)^2)$

$\Rightarrow N(x_i - \mu)^2 \leq N(N-1)\sigma^2 \Rightarrow |x_i - \mu| / \sigma \leq \sqrt{(N-1)}$ ∎

Bizim tasarımda: $N = 48 \rightarrow \sqrt{47} \approx 6{,}86 < 127/16 \approx 7{,}94$ (INT8 Q3.4'ün en büyüğü). Kalibrasyon gerekmez; $\varepsilon > 0$ sınırı daha da küçültür.

Softmax’ta max’ı çıkarmak neden serbest, tek bölme neden yeter?

Kısa cevap

Pay ve payda aynı çarpanı taşır, sadeleşir. Bölme doğrusal olduğu için toplamın dışına çıkar.

Gerekçe

$$e^{(s_j - m)} / \Sigma_i e^{(s_i - m)} = e^{-m} e^{s_j} / (e^{-m} \Sigma_i e^{s_i}) = \mathrm{softmax}(s)_j$$

$$y = \Sigma_j (e_j / E) v_j = (1/E) \cdot \Sigma_j e_j v_j , E = \Sigma_j e_j$$

$$p_j \geq 0, \Sigma p_j = 1 \Rightarrow \min_j v_j \leq y \leq \max_j v_j$$

Son satır: çıktı değerlerin dışbükey birleşimi, aralığı aşamaz. En büyük terim $e = 1$ olduğundan $E \geq 2^{15}$: sıfıra bölme de yok.

Bizim tasarımda: baş başına bir kez $R = \lfloor 2^{32}/E \rfloor$, sonra her boyut için $y = (A \cdot R + 2^{31}) \gg 32$; çıkış ölçeği değerlerin ölçeğiyle aynı.

“Bit-exact” ne demek; “yeterince yakın” neden yetmez?

Kısa cevap

Donanımın her logiti, spesifikasyonun tamsayısıyla birebir aynı. Tek bir farklı bit bir hatadır.

Üç neden

1. Spesifikasyon deterministik tamsayı aritmetiği: tek bir doğru cevap var. Bir tolerans için gerekçe yok.
2. Tolerans sistematik hataları gizler: “yarıya yuvarla” yerine “aşağı yuvarla” ±1 LSB fark üretir, testi geçer, ama katmanlar ve adımlar boyunca birikir.
3. Hata yerini gösterir: her alt katmandan sonraki artık akış izlenir; ilk farklı değer, hatalı birimi işaret eder.

> **Bizden bir örnek:** AXI tezgâhı yükselen kenarda örnekleyince sonuç simülatörün olay sırasına bağlıydı. Düşen kenarda örnekleyen yarışsız bir modelle bu belirsizlik kalktı.

İsimlerde GRU neden önde? Transformer ne zaman kazanır?

Kısa cevap

16 harflik bir geçmişi sabit boyutlu bir durum yeterince taşır. Bağlam uzayınca dikkatin her tokena doğrudan erişimi fark yaratır.

Veri (ön sonuç, tek tohum)

Model	İsimler	Hikâyeler	Çevrim
GRU	**1,956**	1,610	28 861
GPT-2	2,000	1,438	40 236
Llama 2	2,003	**1,346**	48 852

Test kaybı (nat/token). İsimler: 50k, $T \le 16$. Hikâyeler: kendi eğittiğimiz 260k modeller, $T = 256$; çevrim bu modellerde, 256 konum ortalaması.

> **Bedeli:** GRU'nun çevrimi konumdan bağımsız; dikkatinki her katmanda $H \cdot 2(t+1)$ ile doğrusal artar. Kalite kazancı ve donanım maliyeti aynı tabloda okunur.

8 · NEREDEYİZ

Yapılanlar, kalanlar ve sınırlar

Tamam (simülasyonda)

- minGPT hızlandırıcısı, 729/729
- 9 mimari ve stories260K, tek motor
- İspatlı sınırlı tamsayı çekirdekler
- Doğrulama zinciri ve sürekli entegrasyon
- Çevrim modeli, *P* taraması, Yosys
- Kart sürücüsü, makale ve rapor taslağı

Kalan

- Vivado: 100 MHz zamanlama, kesin kaynak
- PYNQ-Z2'de gecikme, ARM ile karşılaştırma
- Güç ve token başına enerji
- Eğitim kampanyası: LR taraması, çok tohum, 12k ve 200k ölçekleri
- Sonuç bölümlerinin yazımı

Sınırlar

- Küçük ölçek: 12k–260k parametre; bulgular büyük modellere aktarılmayabilir
- Kaynak sayıları şimdilik Yosys tahmini
- Aile sonuçları tek tohumlu ön sonuç

ÖZET

1 · minGPT’nin her adımını formülden koda, koddan tamsayıya, tamsayıdan RTL’ye taşıdık.

2 · Dokuz mimariyi tek bir motorda, bit bit doğrulanmış biçimde çalıştırıyoruz.

3 · Her mimari fikrin kalitesi ve donanım maliyeti artık aynı tabloda.

Teşekkürler. Sorular?

github.com/Muratatceken/mingpt-fpga